PHD PROGRAM

Applied Data Science & Artificial Intelligence



UNIVERSITÀ DEGLI STUDI DI TRIESTE

Counterfactual Explanations

(and how to find them)

Counterfactual Explanations



Introduction to Counterfactuals Explanations

Most ML models are *black-box* models

Humanly impossible to directly understand the decision process adopted

Counterfactual explanations suggest *what should be different in the input* instance *to change the outcome* of an AI system

Highlight cause-effect relations
Similar to how children learn

Introduction to Counterfactuals Explanations (cont'd)

Highest level of Pearl's interpretability scale

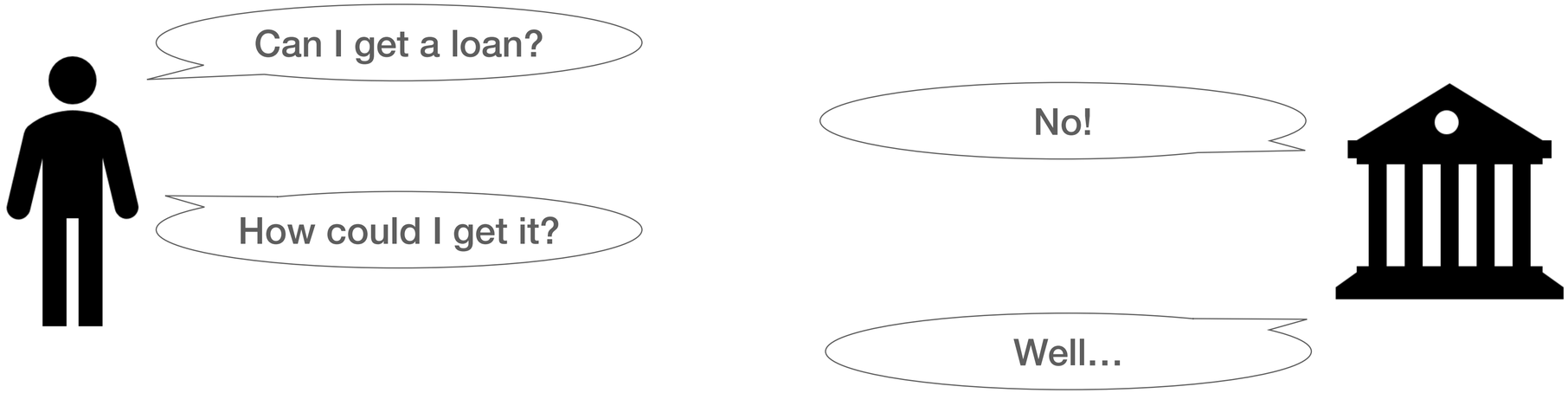
Automatically explore *«what-if»* scenarios

Favour creative problem solving
(Might) Enable actionable recourse

Might suggest changing non actionable features

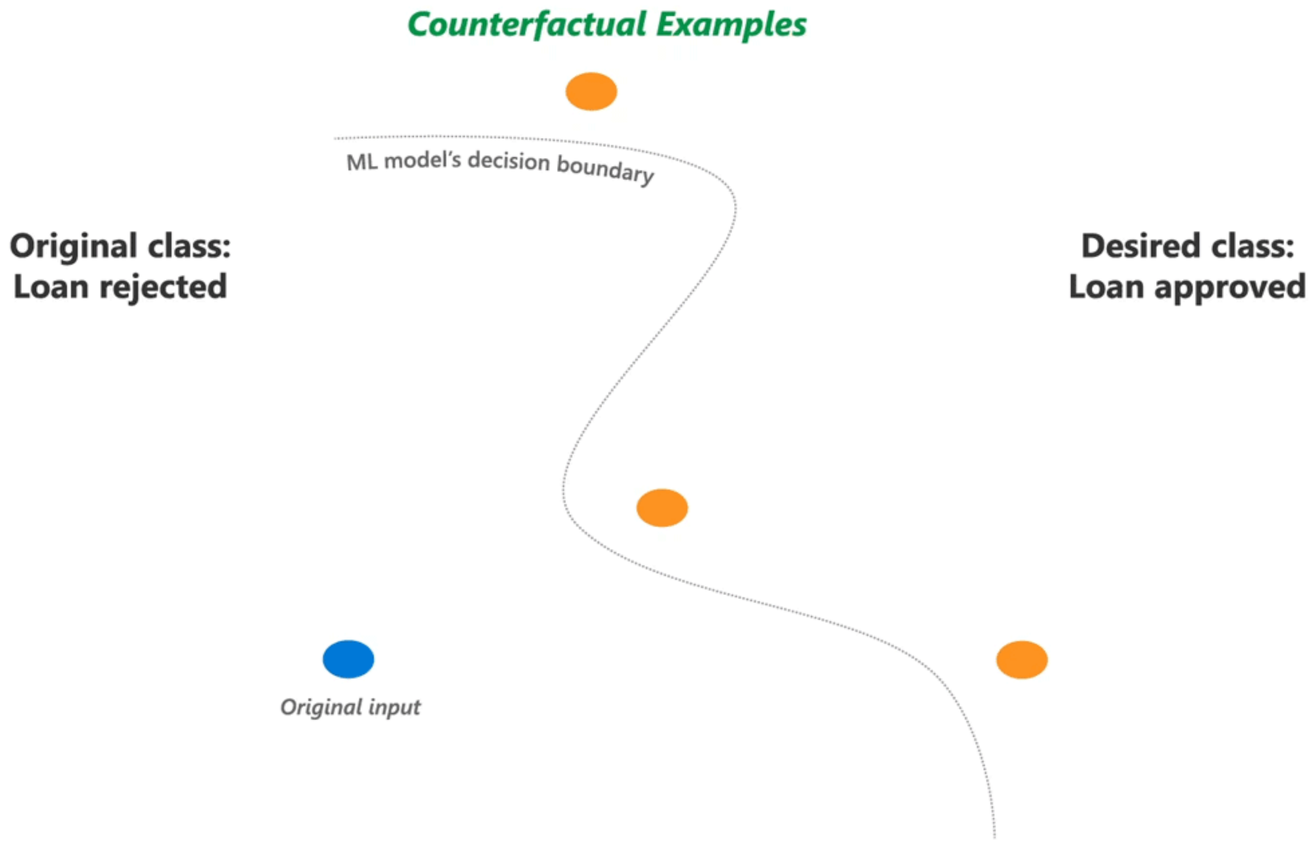
Help unveil *biases* in the system

A common example

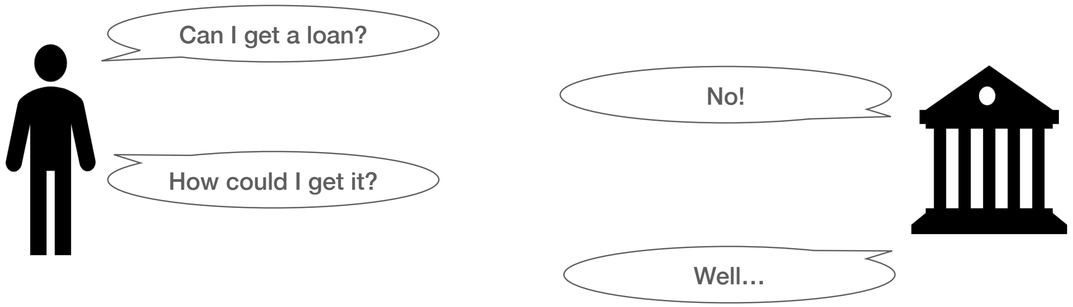


Here’s where counterfactuals come into play

A motivating example



A motivating example



1. You could earn 5000$ more
2. You could earn 2000$ more and have no debts
3. You could *have no children*
4. You could *be younger* and *have higher education degree*
5. You could be of a *different ethnicity*



Definition 1: *counterfactual explanation*

Given a classifier b that outputs the decision $y = b(x)$ for an instance x, a **counterfactual explanation** consists of an instance x' such that the decision for b on x' is different from y, i.e., $b(x') \neq y$, and such that the difference between x and x' is minimal.

- b is a black-box model
- x and x' consist of a set of m attribute-value pairs $x_i = (a_i, v_i)$
- The not null difference between x and x' reveals what should have been different in x for having a different outcome
- Minimality can refer to *sparsity* (few changes) or *similarity/proximity* (small changes)
- A counterfactual explanation can be a single example or a set of examples

Properties of counterfactual explanations

- Validity: $b(x') \neq b(x)$
- Minimality (sparsity): $\nexists x''$ s.t. $|\delta_{x,x''}| < |\delta_{x,x'}|$
- Similarity (proximity): $d(x, x') < \varepsilon$
- Plausibility: feature values in x' are coherent with those in X
- Discriminative power: looking at x and x' also as humans we would have classified $b(x) = y$ and *b(x') ≠ y* due to the differences $\delta_{x,x'}$ between x and x'
- Actionability: all the differences between x and x' refer to actionable features
- Causality: x' respects all known causalities among the features
- Diversity: an explanation is formed by diverse counterfactuals

Definition 2: *counterfactual explainer*

A **counterfactual explainer** is a function f_k that takes as input a classifier b, a set X of known instances, and a given instance of interest x, and with its application $C = f_k(x, b, X)$ returns a set $C = \{x'_1, \ldots, x'_h\}$ of $h \leq k$ valid counterfactual examples where k is the number of counterfactuals required.

- Most of the counterfactual explainers in the literature return a single valid counterfactual
- If $C = \varnothing$ the explainer was not able to find any valid counterfactual

Properties of counterfactual explainers

- Efficiency: fast enough to be used in real life applications
- Stability: given two similar instances x_1 and x_2, the sets of returned counterfactuals C_1 and C_2 should be similar
- Fairness: changes in x' related to demographic groups (or similar sensitive features) do not change the validity of the outcome

Some categorisations

- Model specific / model agnostic explainers
- Domains of applicability: tabular data, images, textual data
- Endogenous / exogenous explainers (sample test data or novel data points)

How to find counterfactuals



Finding counterfactuals

- **Optimisation**

  Defines a loss function

  Diverse Counterfactual Explanations (DiCE)

- **Heuristic Search Strategy**

  Minimisation of a cost function

  Counterfactual Local Explanations via Regression (CLEAR)

- **Instance-Based**

  Select most similar examples from a dataset

  Feasible and Actionable Counterfactual Explanations (FACE)

- **Decision Tree**

  Approximate the black-box behaviour with a decision tree (and exploit it to find counterfactuals)

  Local Rule-based Explainer (LORE)

Finding counterfactuals

- **Optimisation**

  Defines a loss function

  Diverse Counterfactual Explanations (DiCE)

- **Heuristic Search Strategy**

  Minimisation of a cost function

  Counterfactual Local Explanations via Regression (CLEAR)

- **Instance-Based**

  Select most similar examples from a dataset

  Feasible and Actionable Counterfactual Explanations (FACE)

- **Decision Tree**

  Approximate the black-box behaviour with a decision tree (and exploit it to find counterfactuals)

  Local Rule-based Explainer (LORE)

Diverse Counterfactual Explanations (DiCE)

- Diversity as key component for actionability → some attributes are easier to change
- Trade-off between diversity and proximity
- Relies on the minimisation of a loss followed by a filtering approach to ensure causal constraints are satisfied
- Applicable to any static differentiable ML (binary) classifier
- Evaluation metrics and boundary approximation.

DiCE – loss

$$L(x) = \underset{x_1,\ldots,x_k}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \operatorname{yloss}(b(x_i), y) + \frac{\lambda_1}{k} \sum_{i=1}^{k} \operatorname{dist}(x_i,\ x) - \lambda_2 \operatorname{diversity}(x_1, \ldots,\ x_k)$$

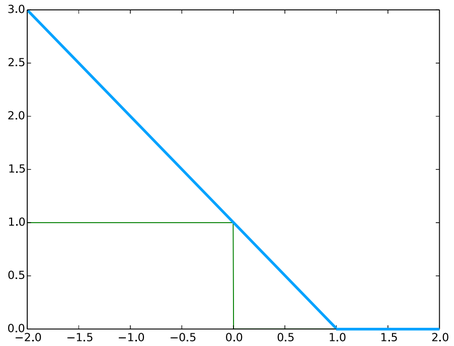
- Loss minimised via Gradient Descent (Adam optimiser)
- Maximum 5000 steps
- Initialise all x_i randomly

DiCE – loss

$$L(x) = \underset{x_1,\ldots,x_k}{\operatorname{argmin}} \frac{1}{k}\sum_{i=1}^{k} \text{yloss}(b(x_i), y) + \frac{\lambda_1}{k}\sum_{i=1}^{k} \text{dist}(x_i,\ x) - \lambda_2 \text{diversity}(x_1, \ldots,\ x_k)$$

- yloss pushes the counterfactual towards validity
- If we rely on a distance function, we might encourage large changes (lowering feasibility)

  → we only need to cross the classification threshold
- **hinge-loss function**
  - Zero penalty when far *enough* (more than a threshold) from the border
  - Penalty proportional to the distance from the border if within the threshold but correct
  - Heavier penalty if wrong



DiCE – loss

$$L(x) = \underset{x_1,\ldots,x_k}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \operatorname{yloss}(b(x_i), y) + \frac{\lambda_1}{k} \sum_{i=1}^{k} \operatorname{dist}(x_i, x) - \lambda_2 \operatorname{diversity}(x_1, \ldots, x_k)$$

Continuous variables

$$\operatorname{dist}_{\text{cont}}(x', x) = \frac{1}{d_{\text{cont}}} \sum_{p=1}^{d_{\text{cont}}} \frac{|x'^p - x^p|}{\text{MAD}_p}$$

- Mean of feature-wise distances
- Divide by MAD (mean abs. deviation) to capture relative prevalence of observing a feature at a given value

Categorical variables

$$\operatorname{dist}_{\text{cat}}(x', x) = \frac{1}{d_{\text{cat}}} \sum_{p=1}^{d_{\text{cat}}} I(x'^p \neq x^p)$$

- Does not capture the difficulty of changing a value of a particular feature

DiCE – loss

$$L(x) = \underset{x_1,\ldots,x_k}{\operatorname{argmin}} \frac{1}{k} \sum_{i=1}^{k} \operatorname{yloss}(b(x_i), y) + \frac{\lambda_1}{k} \sum_{i=1}^{k} \operatorname{dist}(x_i,\ x) - \lambda_2 \operatorname{diversity}(x_1, \ldots,\ x_k)$$

- Diversity via Determinantal Point Processes: point processes in which points are sampled under a repulsion force pushing points away from each other.
- Determinant of the Kernel matrix given the counterfactuals

$$\operatorname{diversity}(x_1, \ldots,\ x_k) = \det(\boldsymbol{K}) \qquad K_{i,j} = \frac{1}{1 + \operatorname{dist}(x_i x_j)}$$

DiCE – validation

$$\%\texttt{Valid-CFs} = \frac{|\{\text{unique instances in } C \text{ s.t. } f(c) > 0.5\}|}{k}$$

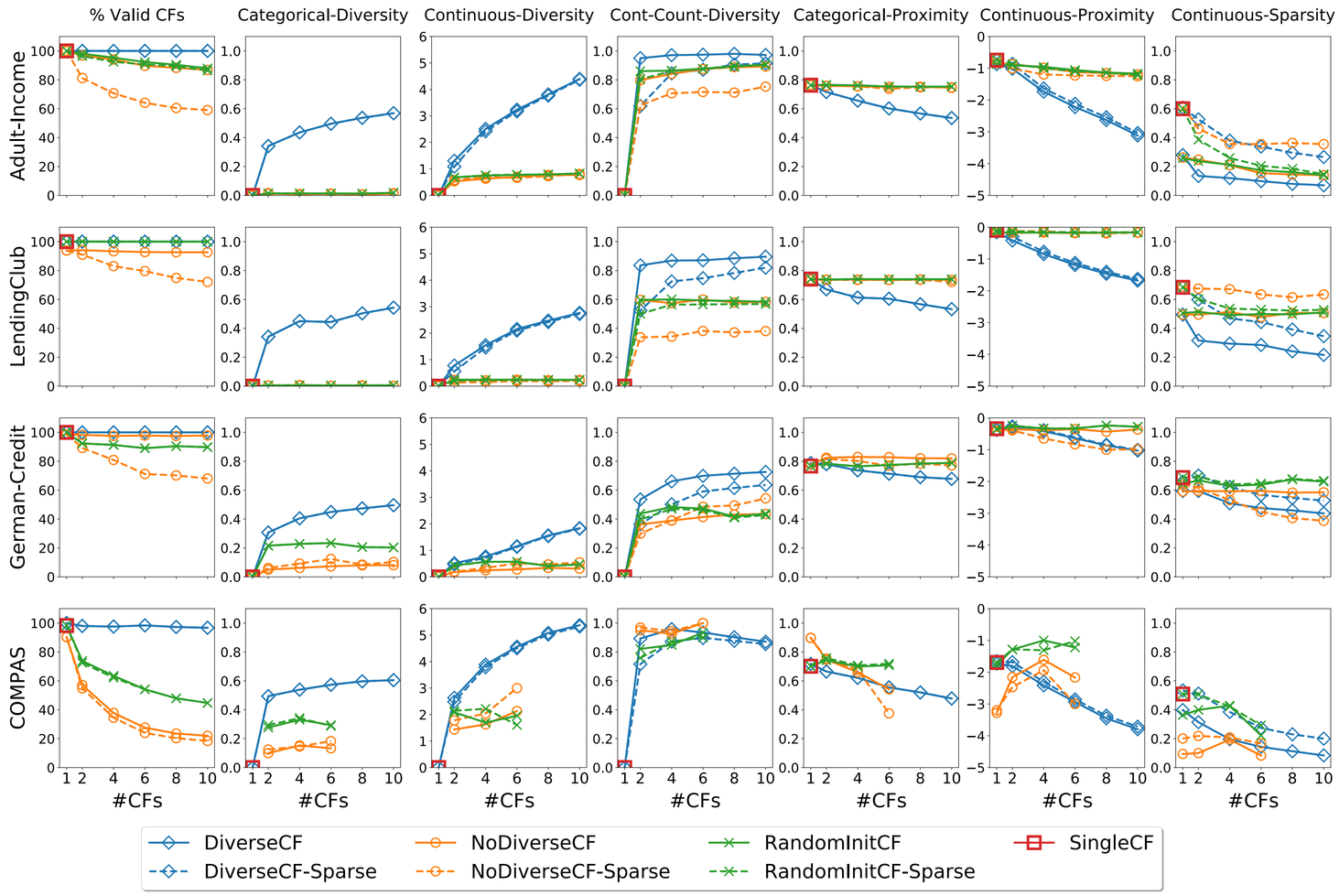
$$\texttt{Continuous-Proximity} : -\frac{1}{k}\sum_{i=1}^{k} \text{dist_cont}(\boldsymbol{c}_i, \boldsymbol{x}),$$

$$\texttt{Categorical-Proximity} : 1 - \frac{1}{k}\sum_{i=1}^{k} \text{dist_cat}(\boldsymbol{c}_i, \boldsymbol{x}),$$

$$\texttt{Sparsity} : 1 - \frac{1}{kd}\sum_{i=1}^{k}\sum_{l=1}^{d} 1_{[\boldsymbol{c}_i^l \neq \boldsymbol{x}_i^l]}$$

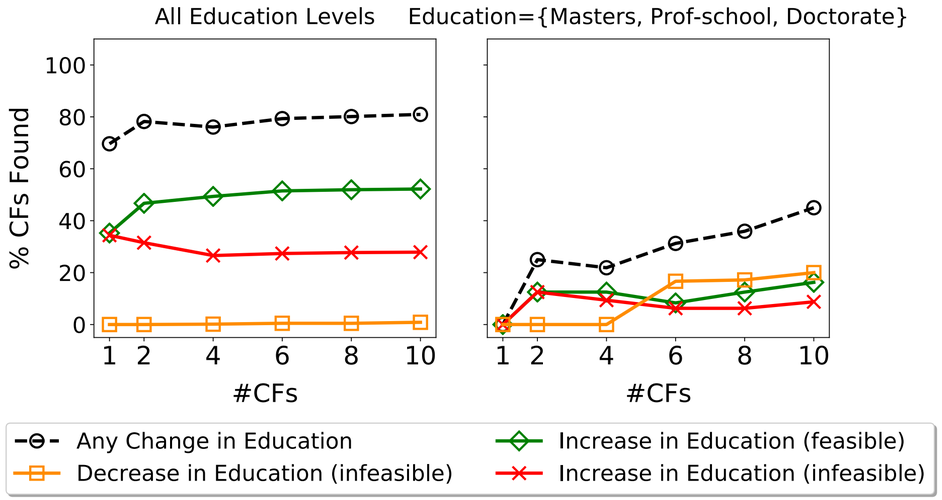
$$\texttt{Diversity} : \Delta = \frac{1}{C_k^2}\sum_{i=1}^{k-1}\sum_{j=i+1}^{k} \text{dist}(\boldsymbol{c}_i, \boldsymbol{c}_j),$$

$$\texttt{Count-Diversity} : \frac{1}{C_k^2 d}\sum_{i=1}^{k-1}\sum_{j=i+1}^{k}\sum_{l=1}^{d} 1_{[\boldsymbol{c}_i^l \neq \boldsymbol{c}_j^l]}$$



DiCE – feasibility

- User can specify unmodifiable features, treated as constants
- Filtering step after the counterfactuals are found: greedily try to revert changes to continuous variables close to original value.
- User provides pairs of features and the direction of causal change between them (e.g., age and education level). Post hoc filtering of counterfactuals according to such constraints



- Education ↑ ⇒ Age ↑, Education↓

DiCE – example

Adult	HrsWk	Education	Occupation	WorkClass	Race	AgeYrs	MaritalStat	Sex
Original input (outcome: <=50K)	45.0	HS-grad	Service	Private	White	22.0	Single	Female
Counterfactuals (outcome: >50K)	—	Masters	—	—	—	65.0	Married	Male
	—	Doctorate	—	Self-Employed	—	34.0	—	—
	33.0	—	White-Collar	—	—	47.0	Married	—
	57.0	Prof-school	—	—	—	—	Married	—

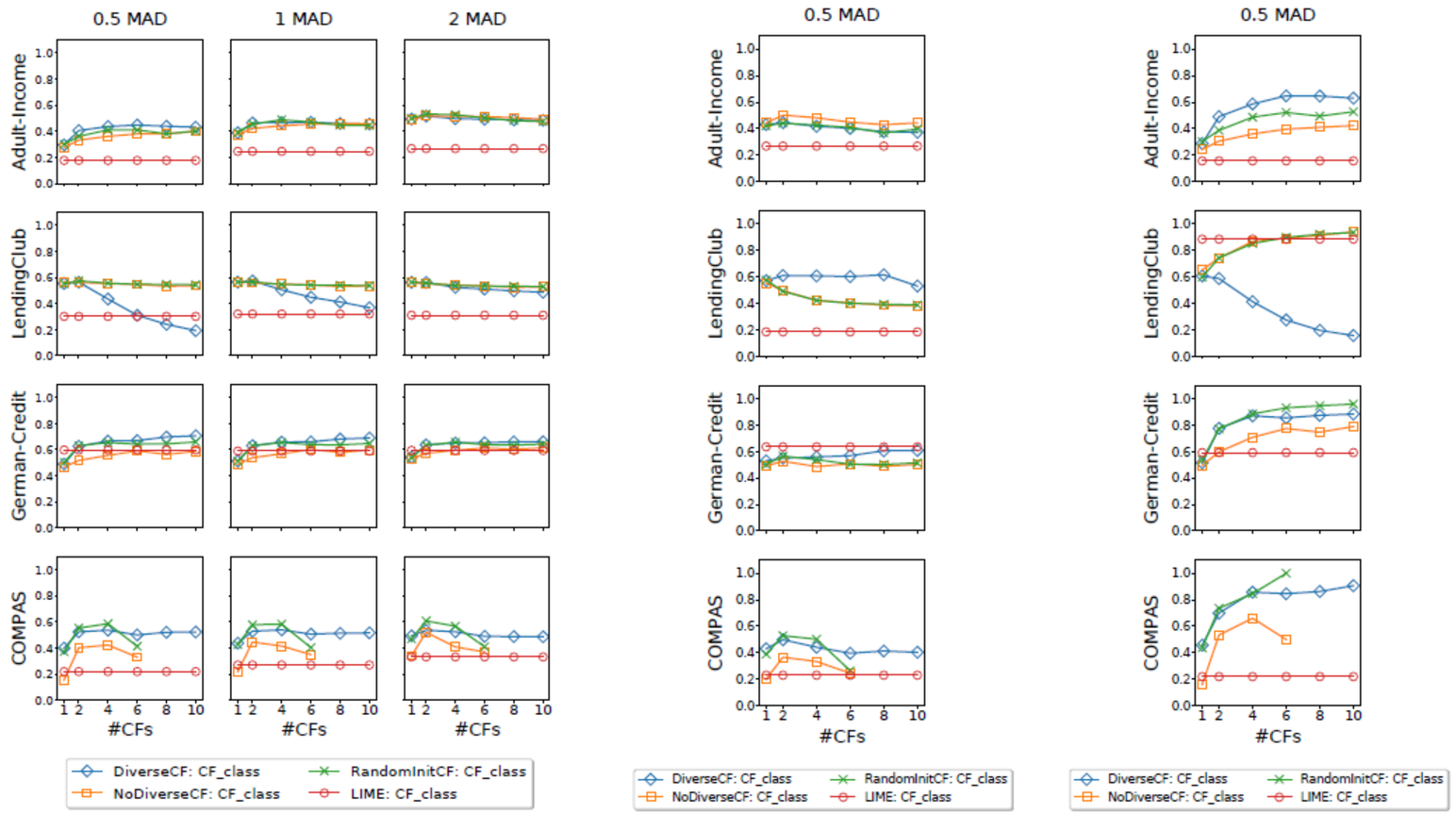
LendingClub	EmpYrs	Inc$	#Ac	CrYrs	LoanGrade	HomeOwner	Purpose	State
Original input (outcome: Default)	7.0	69996.0	4.0	26.0	D	Mortgage	Debt	NY
Counterfactuals (outcome: Paid)	—	61477.0	—	—	B	—	Purchase	—
	10.0	83280.0	1.0	23.0	A	—	—	TX
	10.0	69798.0	—	40.0	A	—	—	—
	10.0	130572.0	—	—	A	Rent	—	—

COMPAS	PriorsCount	CrimeDegree	Race	Age	Sex
Original input (outcome: Will Recidivate)	10.0	Felony	African-American	>45	Female
Counterfactuals (outcome: Won't Recidivate)	—	—	Caucasian	—	—
	0.0	—	—	—	Male
	0.0	—	Hispanic	—	—
	9.0	Misdemeanor	—	—	—

Table 3: Examples of generated counterfactuals in `Adult-Income`, `LendingClub` and `COMPAS` datasets.

DiCE – local decision boundary approximation

- Build a 1-NN classifier from the local instance and the counterfactuals.
- Test it against LIME, in terms of accuracy on data sampled “close to the decision boundary”.



(a) F1 score of the counterfactual class. (b) Precision. (c) Recall.

Figure 3: Performance of 1-NN classifiers learned from counterfactuals at different distances from the original input. `DiverseCF` outperforms LIME and baseline CF methods in F1 score on correctly predicting the counterfactual class, except in `LendingClub` dataset. For `Adult-Income` and `COMPAS` datasets, both precision and recall is higher for `DiverseCF` compared to `LIME`.

Finding counterfactuals

- **Optimisation**

  Defines a loss function

  Diverse Counterfactual Explanations (DiCE)

- **Heuristic Search Strategy**

  Minimisation of a cost function

  Counterfactual Local Explanations via Regression (CLEAR)

- **Instance-Based**

  Select most similar examples from a dataset

  Feasible and Actionable Counterfactual Explanations (FACE)

- **Decision Tree**

  Approximate the black-box behaviour with a decision tree (and exploit it to find counterfactuals)

  Local Rule-based Explainer (LORE)

Counterfactual Local Explanations via Regression (CLEAR)

- Provides satisfactory explanations → understanding of why an event occurs + changes to features that would have resulted in a different outcome

- Relies on (boundary) *b*-counterfactuals (for the changes) and trains LIME upon them (for an explanation of the prediction)

CLEAR – b-perturbation

Let $min_i(x)$ denote a vector resulting from applying a minimum change to the value of one feature i in x such that $b(min_i(x)) \neq b(x)$.

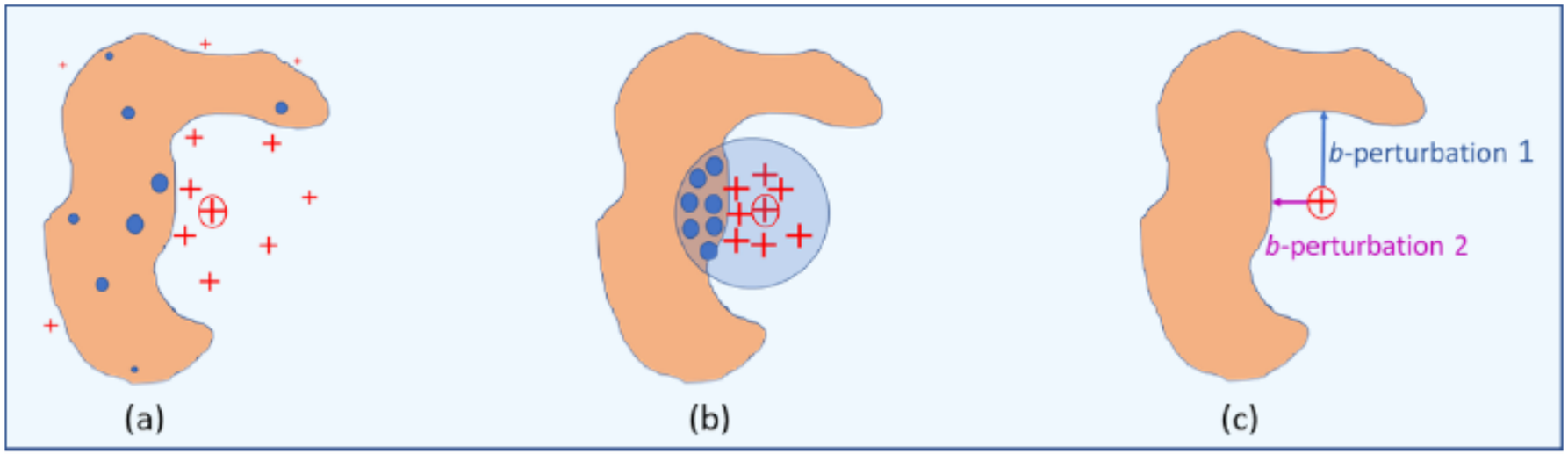
Let $v_i(x)$ denote the value of feature i in x.

A **b-perturbation** is defined as the change in value of feature $i, v_i(min_i(x)) - v_i(x)$.

- p features, k classes → there are at most $(p \times k - 1)$ b-perturbations

CLEAR – toy example

Figure 1: Toy example of a machine learning function represented by tan/blue background. The circled cross is x whose prediction is to be explained. The other crosses are synthetic observations. (a) LIME uses all synthetic observations in each regression (15,000 in this paper) with weights decreasing with distance from x. (b) CLEAR selects a balanced subset of $\approx$ 200 synthetic observation. (c) shows the corresponding b-perturbations.



CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature* f **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature* f **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

Sampling from a uniform distribution for continuous variables, and in proportion to the frequencies for categorical variables.
Samples are labelled passing through m (the model, we called it b).

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature* f **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

For each feature i, a separate one-dimensional search is performed

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature f* **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

Algorithm 2: Balanced_Neighbourhood

input : S (synthetic dataset), $\boldsymbol{x}$,m
  b_1, b_2 (margins around decision boundary)
output: N (neighbourhood dataset)
$n \leftarrow 200$
for $s_i \in S$ **do**
  $d_i \leftarrow$ Euclidean_Distance $(s_i, \boldsymbol{x})$
  $y_i \leftarrow$m(s_i)
end
$N_1 \leftarrow n/3$ members of $\{S\}$ with lowest d_i s.t. $0 < y_i \leq b_1$
$N_2 \leftarrow n/3$ members of $\{S\}$ with lowest d_i s.t. $b_1 < y_i \leq b_2$
$N_3 \leftarrow n/3$ members of $\{S\}$ with lowest d_i s.t. $b_2 < y_i \leq 1$
return $N \leftarrow N_1 \cup N_2 \cup N_3$

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature* f **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

Step-wise regression on the neighbourhood dataset under the constraint that the curve must go through x

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature f* **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

Using the regressor and explicitly finding roots of the equation

CLEAR – algorithm

Algorithm 1: CLEAR Algorithm

input : t (training data), $\boldsymbol{x}$,m,T
output: expl (set of explanations)
$S \leftarrow$ Generate_Synthetic_Data($\boldsymbol{x}$,t,m)
for *each target class tc* **do**
  for *each feature f* **do**
    $w \leftarrow$ Find_Counterfactuals($\boldsymbol{x}$,m)
  end
  $N_{tc} \leftarrow$ Balanced_Neighbourhood(S, $\boldsymbol{x}$, m)
  Optional: $N_{tc} \leftarrow N_{tc} \cup w$
  $r \leftarrow$Find_Regression_Equations(N_{tc},$\boldsymbol{x}$)
  $w' \leftarrow$ Estimate_Counterfactuals(r,$\boldsymbol{x}$)
  $e \leftarrow$ Calculate_Fidelity(w,w',T)
  return $\text{expl}_{tc} =< w, w', r, e >$
end

fidelity error = | estimated b-perturbation – b-perturbation |

CLEAR – report example

Figure 2: Excerpt from a CLEAR *b*-counterfactual report. In this example CLEAR uses multiple regression to explain a single prediction generated by an MLP model trained on the Pima dataset

CLEAR Report: PIMA dataset

Prediction to be explained: Observation 2 has 0.72 probability of diabetes

***b*-counterfactuals**

feature	input value	actual *b*-counterfactual value
Glucose	0.24	-1.08
BMI	0.65	-0.89
Age	0.36	-0.92

Regression

prediction = 0.44 - 0.018 BloodPressure + 0.16 BMI + 0.015 Pregnancies - 0.013 SkinThickness + 0.23 Glucose - 0.12 Insulin + 0.061 DiabFunction + 0.23 Age - 0.11 Age^2 - 0.071 BMI^2 + 0.12 (Insulin*Age)

feature	estimated *b*-counterfactual value	fidelity error
Glucose	-0.72	0.37
BMI	-0.73	0.17
Age	-0.74	0.18

Finding counterfactuals

- **Optimisation**

  Defines a loss function

  Diverse Counterfactual Explanations (DiCE)

- **Heuristic Search Strategy**

  Minimisation of a cost function

  Counterfactual Local Explanations via Regression (CLEAR)

- **Instance-Based**

  Select most similar examples from a dataset

  Feasible and Actionable Counterfactual Explanations (FACE)

- **Decision Tree**

  Approximate the black-box behaviour with a decision tree (and exploit it to find counterfactuals)

  Local Rule-based Explainer (LORE)

Feasible and Actionable Counterfactual Explanations (FACE)

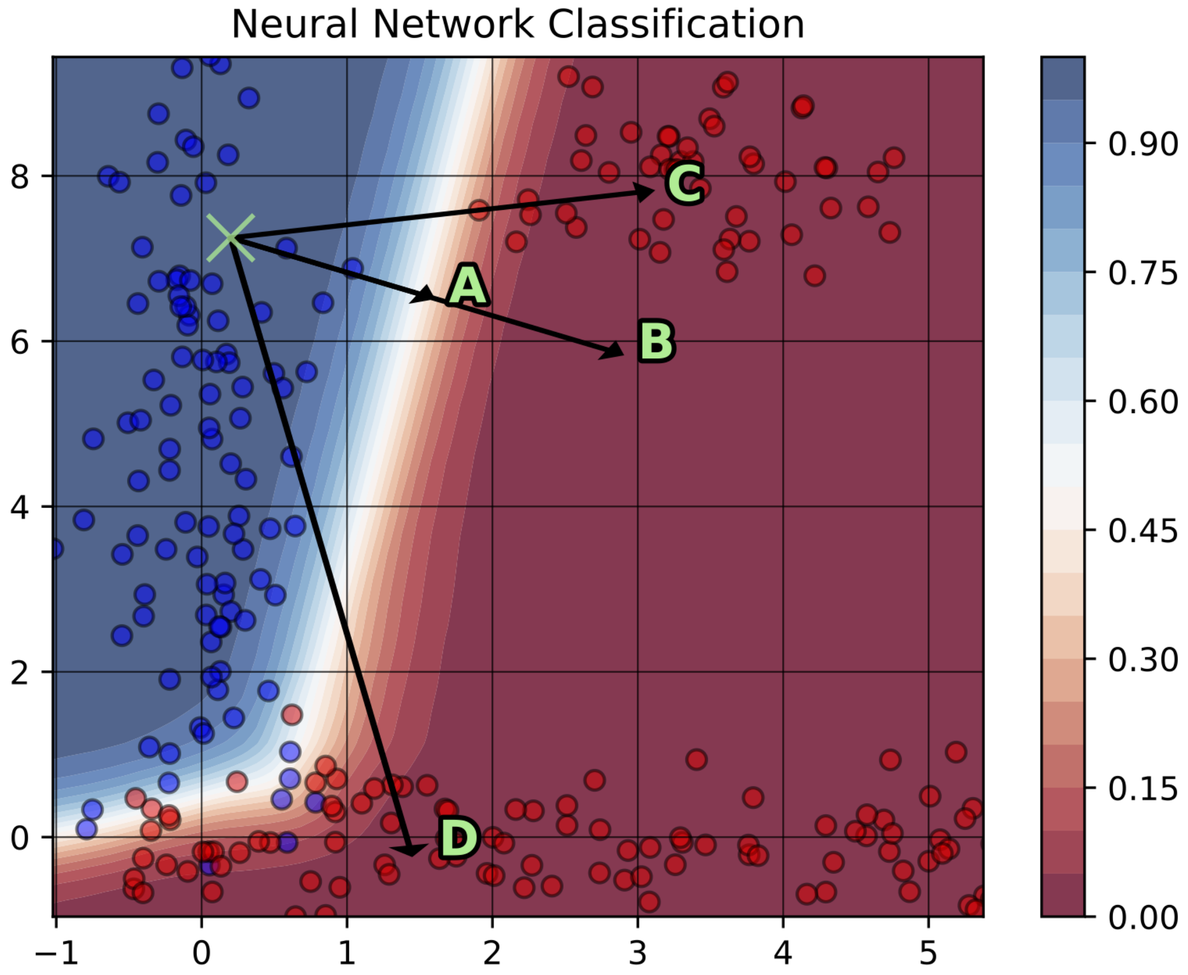
- Criticises state-of-the-art approaches → generated counterfactuals may not be representative of the underlying data distribution (prescribe unachievable goals)
  + they may not be based on a feasible path between the current state and the target one (making actionable recourse infeasible)

- Idea of finding counterfactuals supported by the *feasible paths of change* (density path ≥ T)
  → based on shortest path distances on weighted graphs where weights encode metric information to approximate path integrals.

$$\hat{\mathcal{D}}_{f,\gamma} = \sum_{i=1}^{N} f_p\left(\frac{\gamma(t_{i-1}) + \gamma(t_i)}{2}\right) \cdot \|\gamma(t_{i-1}) - \gamma(t_i)\|.$$

- Endogenous and data-agnostic method

Feasible and Actionable Counterfactual Explanations (FACE)



- A and B are out of distribution
- C is on manifold but the path to C requires changes which pass to low density regions
- D is on manifold and the path to it is meaningful

FACE – algorithm

Algorithm 1 FACE Algorithm

Input: Data $X \in \mathbb{R}^p$, density estimator $\hat{p} : \chi \to [0, 1]$, probabilistic predictor $b : \chi \to [0, 1]^k$, distance function $d : \chi \times \chi \to \mathbb{R}_{\geq 0}$, distance threshold $\varepsilon > 0$, weight function $w : \chi \times \chi \to \mathbb{R}_{\geq 0}$, conditions function $c : \chi \times \chi \to \{\mathrm{T}, \mathrm{F}\}$, target class $\hat{y}$

Output: graph $\mathcal{G}(V, E, W)$, candidate targets T

▷ Construct the graph $\mathcal{G}$
for all $x_i, x_j \in X$ **do**
    if $d(x_i, x_j) < \varepsilon$ **and** $c(x_i, x_j) = \mathrm{T}$ **then**
        $i \sim j$
        $w_{i,j} = w(x_i, x_j)$
    end if
end for
▷ Get candidate targets T
$T = \emptyset$
for all $x_i \in X$ **do**
    if $b(x_i)[\hat{y}] \geq t_p$ **and** $\hat{p}(x_i) \geq t_d$ **then**
        $T = T \bigcup \{x_i\}$
    end if
end for
return $\mathcal{G}, T$

FACE – algorithm

Algorithm 1 FACE Algorithm

Input: Data $X \in \mathbb{R}^p$, density estimator $\hat{p} : \chi \to [0, 1]$, probabilistic predictor $b : \chi \to [0, 1]^k$, distance function $d : \chi \times \chi \to \mathbb{R}_{\geq 0}$, distance threshold $\varepsilon > 0$, weight function $w : \chi \times \chi \to \mathbb{R}_{\geq 0}$, conditions function $c : \chi \times \chi \to \{\mathrm{T}, \mathrm{F}\}$, target class $\hat{y}$

Output: graph $\mathcal{G}(V, E, W)$, candidate targets T

▷ Construct the graph $\mathcal{G}$
for all $x_i, x_j \in X$ **do**
  if $d(x_i, x_j) < \varepsilon$ **and** $c(x_i, x_j) = \mathrm{T}$ **then**
    $i \sim j$
    $w_{i,j} = w(x_i, x_j)$
  end if
end for
▷ Get candidate targets T
$T = \emptyset$
for all $x_i \in X$ **do**
  if $b(x_i)[\hat{y}] \geq t_p$ **and** $\hat{p}(x_i) \geq t_d$ **then**
    $T = T \bigcup \{x_i\}$
  end if
end for
return $\mathcal{G}, T$

The two points must be close “enough” and it should be possible to act the change between them (according to the conditions)

FACE – algorithm

Algorithm 1 FACE Algorithm

Input: Data $X \in \mathbb{R}^p$, density estimator $\hat{p} : \chi \to [0,1]$, probabilistic predictor $b : \chi \to [0,1]^k$, distance function $d : \chi \times \chi \to \mathbb{R}_{\geq 0}$, distance threshold $\varepsilon > 0$, weight function $w : \chi \times \chi \to \mathbb{R}_{\geq 0}$, conditions function $c : \chi \times \chi \to \{\mathrm{T}, \mathrm{F}\}$, target class $\hat{y}$

Output: graph $\mathcal{G}(V, E, W)$, candidate targets T

▷ Construct the graph $\mathcal{G}$
for all $x_i, x_j \in X$ **do**
  if $d(x_i, x_j) < \varepsilon$ **and** $c(x_i, x_j) = \mathrm{T}$ **then**
    $i \sim j$
    $w_{i,j} = w(x_i, x_j)$
  end if
end for
▷ Get candidate targets T
$T = \emptyset$
for all $x_i \in X$ **do**
  if $b(x_i)[\hat{y}] \geq t_p$ **and** $\hat{p}(x_i) \geq t_d$ **then**
    $T = T \bigcup \{x_i\}$
  end if
end for
return $\mathcal{G}, T$

Kernel Density Estimation, *k*-NN, ε-graph

$$w_{ij} = f_{\hat{p}}\left(\frac{\boldsymbol{x}_i + \boldsymbol{x}_j}{2}\right) \cdot \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \qquad (KDE)$$

$$w_{ij} = \tilde{f}\left(\frac{r}{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|}\right) \cdot \|\boldsymbol{x}_i - \boldsymbol{x}_j\|, \quad r = \frac{k}{N \cdot \eta_d} \qquad (k\text{-NN})$$

$$w_{ij} = \tilde{f}\left(\frac{\epsilon^d}{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|}\right) \cdot \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \qquad (\epsilon\text{-graph})$$

$$\text{when} \quad \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \leq \epsilon$$

FACE – algorithm

Algorithm 1 FACE Algorithm

Input: Data $X \in \mathbb{R}^p$, density estimator $\hat{p} : \chi \to [0, 1]$, probabilistic predictor $b : \chi \to [0, 1]^k$, distance function $d : \chi \times \chi \to \mathbb{R}_{\geq 0}$, distance threshold $\varepsilon > 0$, weight function $w : \chi \times \chi \to \mathbb{R}_{\geq 0}$, conditions function $c : \chi \times \chi \to \{\mathrm{T}, \mathrm{F}\}$, target class $\hat{y}$

Output: graph $\mathcal{G}(V, E, W)$, candidate targets T

▷ Construct the graph $\mathcal{G}$
for all $x_i, x_j \in X$ **do**
  if $d(x_i, x_j) < \varepsilon$ **and** $c(x_i, x_j) = \mathrm{T}$ **then**
    $i \sim j$
    $w_{i,j} = w(x_i, x_j)$
  end if
end for
▷ Get candidate targets T
$T = \emptyset$
for all $x_i \in X$ **do**
  if $b(x_i)[\hat{y}] \geq t_p$ **and** $\hat{p}(x_i) \geq t_d$ **then**
    $T = T \bigcup \{x_i\}$
  end if
end for
return $\mathcal{G}, T$

The target must be a counterfactual (with enough confidence) and it must be in a dense enough area (so it is not an outlier)

FACE – algorithm

Algorithm 1 FACE Algorithm

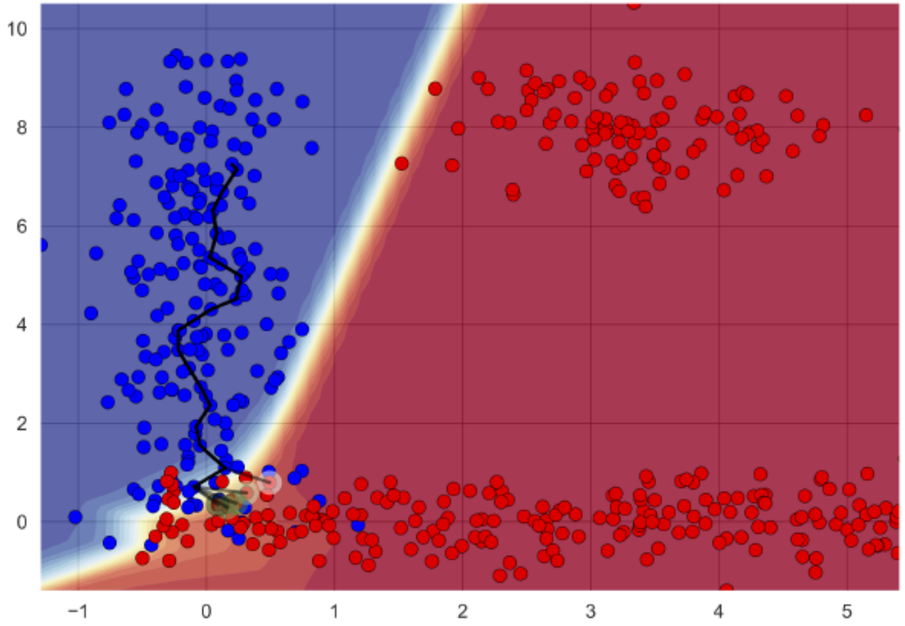
Input: Data $X \in \mathbb{R}^p$, density estimator $\hat{p} : \chi \to [0, 1]$, probabilistic predictor $b : \chi \to [0, 1]^k$, distance function $d : \chi \times \chi \to \mathbb{R}_{\geq 0}$, distance threshold $\varepsilon > 0$, weight function $w : \chi \times \chi \to \mathbb{R}_{\geq 0}$, conditions function $c : \chi \times \chi \to \{\mathrm{T}, \mathrm{F}\}$, target class $\hat{y}$

Output: graph $\mathcal{G}(V, E, W)$, candidate targets T

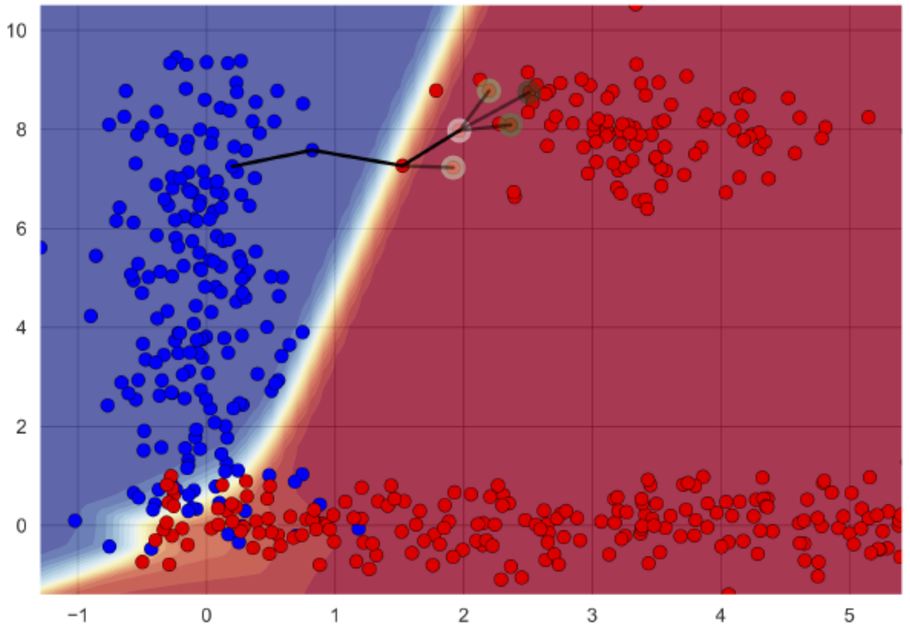
▷ Construct the graph $\mathcal{G}$
for all $x_i, x_j \in X$ **do**
  if $d(x_i, x_j) < \varepsilon$ **and** $c(x_i, x_j) = \mathrm{T}$ **then**
    $i \sim j$
    $w_{i,j} = w(x_i, x_j)$
  end if
end for
▷ Get candidate targets T
$T = \emptyset$
for all $x_i \in X$ **do**
  if $b(x_i)[\hat{y}] \geq t_p$ **and** $\hat{p}(x_i) \geq t_d$ **then**
    $T = T \bigcup \{x_i\}$
  end if
end for
return $\mathcal{G}, T$

Dijkstra algorithm will be used to compute the shortest path between a point and any target on the graph

FACE – example of paths



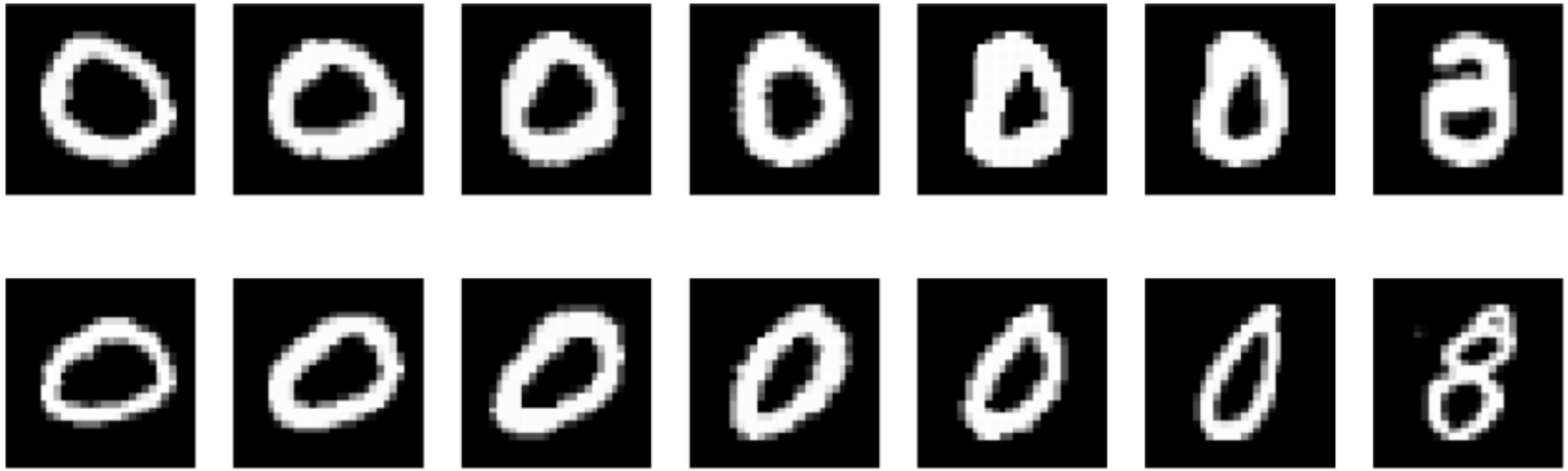
$\epsilon = 0.50$ **distance threshold.**



$\epsilon = 1$ **distance threshold.**

The five shortest paths from a starting data point to a target (counterfactual) data point generated from a graph (edge weights computed using the ε-graph approach).

FACE – transformations example



The FACE «transformation» paths between zero and eight (counterfactual) for two different MNIST images



Thanks for the attention!

References

Pearl, Judea. "Causal inference in statistics: An overview." (2009): 96-146.

Guidotti, Riccardo. "Counterfactual explanations and how to find them: literature review and benchmarking." *Data Mining and Knowledge Discovery* (2022): 1-55.

Mothilal, Ramaravind K., Amit Sharma, and Chenhao Tan. "Explaining machine learning classifiers through diverse counterfactual explanations." *Proceedings of the 2020 conference on fairness, accountability, and transparency*. 2020.

White, Adam, and Artur d'Avila Garcez. "Measurable counterfactual local explanations for any classifier." *arXiv preprint arXiv:1908.03020* (2019).

Poyiadzi, Rafael, et al. "FACE: feasible and actionable counterfactual explanations." *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. 2020.